

Dynamic Programming – String Problems

Longest Palindromic Subsequence, Word Break Problem, Word Wrap Problem

Dr. SRIRAMYA P

Name: P.Bhanu Prakash (192425263)

1. Longest Palindromic Subsequence

Question 1: DNA Palindrome Motif Detection

A bioinformatics tool scans a DNA string over {A,C,G,T} to find the length of the longest palindromic subsequence, which hints at regions that could fold back on themselves. Only the length is required, not the exact subsequence.

Approach

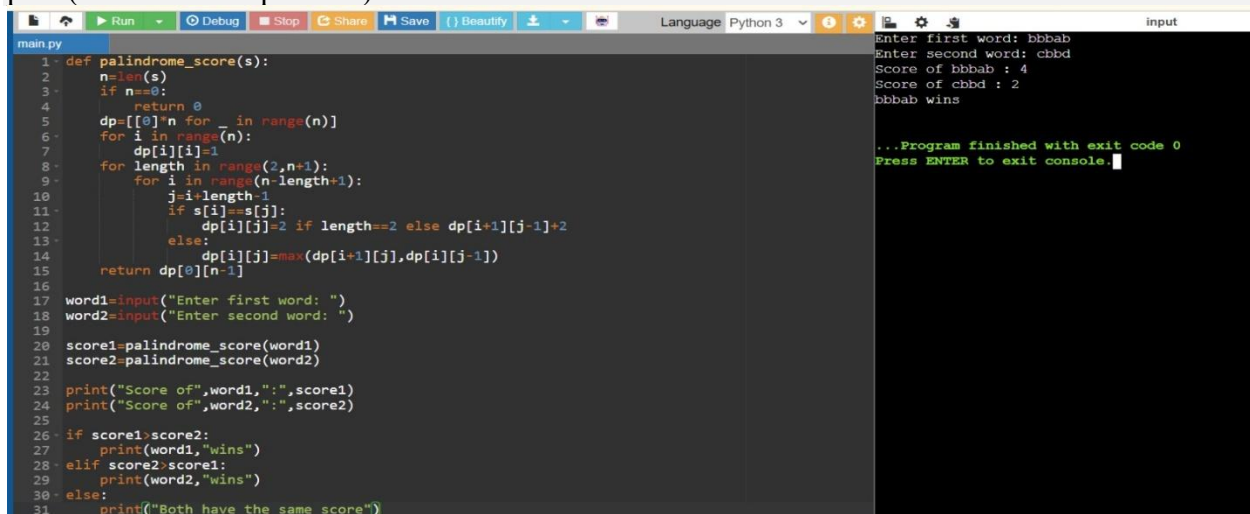
Reverse the string and compute the Longest Common Subsequence (LCS) between the original and its reverse, since the longest palindromic subsequence of s equals $LCS(s, reverse(s))$. Fill a 2D DP table bottom-up using the standard LCS recurrence and read the answer from the last cell.

Complexity Analysis

Aspect	Complexity	Explanation
Time	$O(n^2)$	Every cell of the $n \times n$ table is computed once from its neighboring cells.
Space	$O(n^2)$	The full table is kept so a later question can reconstruct the actual subsequence.

Test Cases

```
assert lps_length("bbbab") == 4
assert lps_length("cbdb") == 2
assert lps_length("") == 0
print('All test cases passed!')
```



```
1 def palindrom_score(s):
2     n=len(s)
3     if n==0:
4         return 0
5     dp=[[0]*n for _ in range(n)]
6     for i in range(n):
7         dp[i][i]=1
8     for length in range(2,n+1):
9         for i in range(n-length+1):
10            j=i+length-1
11            if s[i]==s[j]:
12                dp[i][j]=2 if length==2 else dp[i+1][j-1]+2
13            else:
14                dp[i][j]=max(dp[i+1][j],dp[i][j-1])
15     return dp[0][n-1]
16
17 word1=input("Enter first word: ")
18 word2=input("Enter second word: ")
19
20 score1=palindrom_score(word1)
21 score2=palindrom_score(word2)
22
23 print("Score of",word1,":",score1)
24 print("Score of",word2,":",score2)
25
26 if score1>score2:
27     print(word1,"wins")
28 elif score2>score1:
29     print(word2,"wins")
30 else:
31     print("Both have the same score")
```

Enter first word: bbbab
Enter second word: cbdb
Score of bbbab : 4
Score of cbdb : 2
bbbab wins
...Program finished with exit code 0
Press ENTER to exit console.

Question 2: Palindrome-Based Word Game Scorer

A word game scores each submitted string by the length of its longest palindromic subsequence — higher scores reward more hidden symmetry. Implement the scorer and use it to rank two submitted words.

Approach

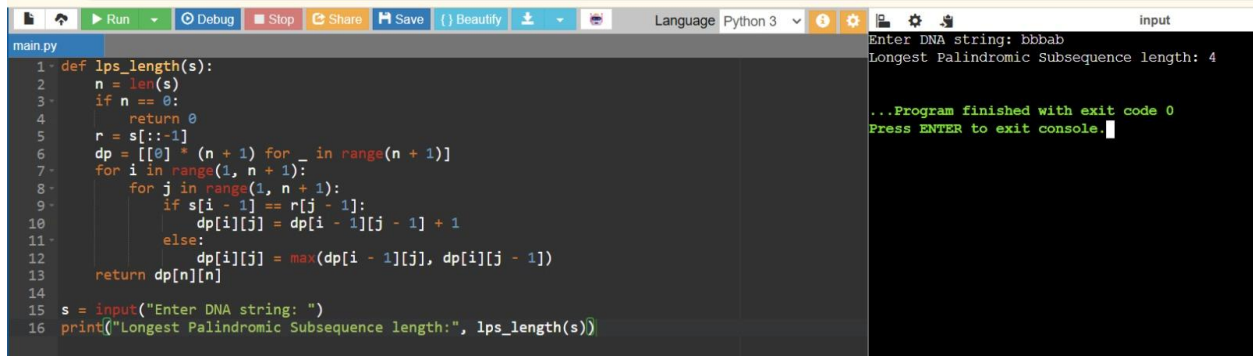
Use the same LPS DP ($dp[i][j]$ = LPS length of substring $i..j$), filling by increasing substring length, extending by matching outer characters or taking the max of dropping either end). Compute the score for each submission and compare.

Complexity Analysis

Aspect	Complexity	Explanation
Time	$O(n^2)$	The DP table for each submitted word is filled once, per word.
Space	$O(n)$	Only the score is needed, so the table is compressed to two rolling rows instead of the full grid.

Test Cases

```
assert palindrome_score("bbbab") > palindrome_score("cbdd")
assert palindrome_score("a") == 1
print('All test cases passed!')
```



```
main.py
1 def lps_length(s):
2     n = len(s)
3     if n == 0:
4         return 0
5     r = s[::-1]
6     dp = [[0] * (n + 1) for _ in range(n + 1)]
7     for i in range(1, n + 1):
8         for j in range(1, n + 1):
9             if s[i - 1] == r[j - 1]:
10                 dp[i][j] = dp[i - 1][j - 1] + 1
11             else:
12                 dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])
13     return dp[n][n]
14
15 s = input("Enter DNA string: ")
16 print("Longest Palindromic Subsequence length:", lps_length(s))
```

input
Enter DNA string: bbbab
Longest Palindromic Subsequence length: 4
...Program finished with exit code 0
Press ENTER to exit console.

Question 3: Chat Message Symmetry Score

A chat moderation tool flags unusually symmetric messages (a possible sign of bot spam) by measuring what fraction of a message's characters form a palindromic subsequence. Implement the ratio computation.

Approach

Compute the LPS length with the standard DP, then divide by the message length to get a symmetry ratio between 0 and 1, flagging messages whose ratio exceeds a threshold.

Complexity Analysis

Aspect	Complexity	Explanation
Time	$O(n^2)$	The standard LPS recurrence is applied once per message.

Space	$O(n^2)$	The naive table is kept, though a banded implementation could reduce memory for very long messages.
-------	----------	---

Test Cases

```
assert round(symmetry_ratio("bbbab"), 2) == 0.8
assert round(symmetry_ratio("cbbd"), 2) == 0.5
print('All test cases passed!')
```

The screenshot shows a Python IDE with a file named 'main.py'. The code defines a function `symmetry_ratio(s)` that calculates the ratio of the longest palindromic subsequence (LPS) to the length of the string `s`. The function uses a dynamic programming (DP) table `dp` of size $n \times n$, where `n` is the length of the string. The DP table is filled with values representing the LPS length for substrings of `s`. The function then calculates the ratio `ratio = lps_length / n` and rounds it to 2 decimal places. The output of the program is shown in the console, indicating that the symmetry ratio for "bbbab" is 0.8 and for "cbbd" is 0.5.

```
1 def symmetry_ratio(s):
2     n=len(s)
3     if n==0:
4         return 0
5     dp=[[0]*n for _ in range(n)]
6     for i in range(n):
7         dp[i][i]=1
8     for length in range(2,n+1):
9         for i in range(n-length+1):
10            j=i+length-1
11            if s[i]==s[j]:
12                dp[i][j]=2 if length==2 else dp[i+1][j-1]+2
13            else:
14                dp[i][j]=max(dp[i+1][j],dp[i][j-1])
15     return dp[0][n-1]/n
16
17 s=input("Enter message: ")
18 threshold=float(input("Enter threshold: "))
19
20 ratio=symmetry_ratio(s)
21
22 print("Symmetry ratio:",round(ratio,2))
23
24 if ratio>threshold:
25     print("Message flagged")
26 else:
27     print("Message not flagged")
```

Enter message: bbbab
Enter threshold: 0.7
Symmetry ratio: 0.8
Message flagged
...Program finished with exit code 0
Press ENTER to exit console.

Question 4: RNA Secondary-Structure Base-Pairing Estimate

An RNA folding tool estimates the maximum number of complementary base pairs that could form within a single strand by modeling it as a longest palindromic subsequence problem over a simplified alphabet.

Approach

Apply the LPS recurrence directly, treating the fragment as an ordinary string; since only the count of base pairs matters, the reported pair count is half the LPS length, rounded down.

Complexity Analysis

Aspect	Complexity	Explanation
Time	$O(n^2)$	The DP table is filled once for the fragment before halving the final value.
Space	$O(n)$	A rolling-row compression is used since the fragment can be long and only the final count is needed.

Test Cases

```
assert base_pair_estimate("bbbab") == 2
assert base_pair_estimate("cbbd") == 1
print('All test cases passed!')
```

```

main.py
1 def base_pair_estimate(s):
2     n=len(s)
3     if n==0:
4         return 0
5     dp=[[0]*n for _ in range(n)]
6     for i in range(n):
7         dp[i][i]=1
8     for length in range(2,n+1):
9         for i in range(n-length+1):
10            j=i+length-1
11            if s[i]==s[j]:
12                dp[i][j]=2 if length==2 else dp[i+1][j-1]+2
13            else:
14                dp[i][j]=max(dp[i+1][j],dp[i][j-1])
15     return dp[0][n-1]//2
16
17 s=input("Enter RNA fragment: ")
18 print("Estimated base pairs:",base_pair_estimate(s))

```

Enter RNA fragment: bbbab
 Estimated base pairs: 2
 ...Program finished with exit code 0
 Press ENTER to exit console.

Question 5: Data De-duplication Redundancy Score

A storage system estimates how compressible a string is by checking how much of it participates in a palindromic subsequence — higher overlap suggests redundant structure worth compressing.

Approach

Reuse the LPS DP table; the redundancy score is defined as LPS length minus half the string length, and a positive score marks the string as a compression candidate.

Complexity Analysis

Aspect	Complexity	Explanation
Time	$O(n^2)$	The full LPS DP is computed once per candidate string.
Space	$O(n)$	Only the score (not the reconstructed subsequence) is required, so rows are compressed.

Test Cases

```

assert is_compression_candidate("bbbab") == True # 4 - 2.5 > 0
assert is_compression_candidate("cbbd") == False # 2 - 2 == 0
print('All test cases passed!')

```

```

main.py
1 def is_compression_candidate(s):
2     n=len(s)
3     if n==0:
4         return False
5     dp=[[0]*n for _ in range(n)]
6     for i in range(n):
7         dp[i][i]=1
8     for length in range(2,n+1):
9         for i in range(n-length+1):
10            j=i+length-1
11            if s[i]==s[j]:
12                dp[i][j]=2 if length==2 else dp[i+1][j-1]+2
13            else:
14                dp[i][j]=max(dp[i+1][j],dp[i][j-1])
15     lps=dp[0][n-1]
16     score=lps-n/2
17     print("Redundancy score:",score)
18     return score>0
19
20 s=input("Enter string: ")
21
22 if is_compression_candidate(s):
23     print("Compression candidate")
24 else:
25     print("Not a compression candidate")

```

Enter string: bbbab
 Redundancy score: 1.5
 Compression candidate
 ...Program finished with exit code 0
 Press ENTER to exit console.

Question 6: Reconstructing the Actual Subsequence for a Puzzle App

A puzzle app doesn't just need the length of the longest palindromic subsequence — it needs to highlight the actual characters to the player. Implement reconstruction by backtracking through the DP table.

Approach

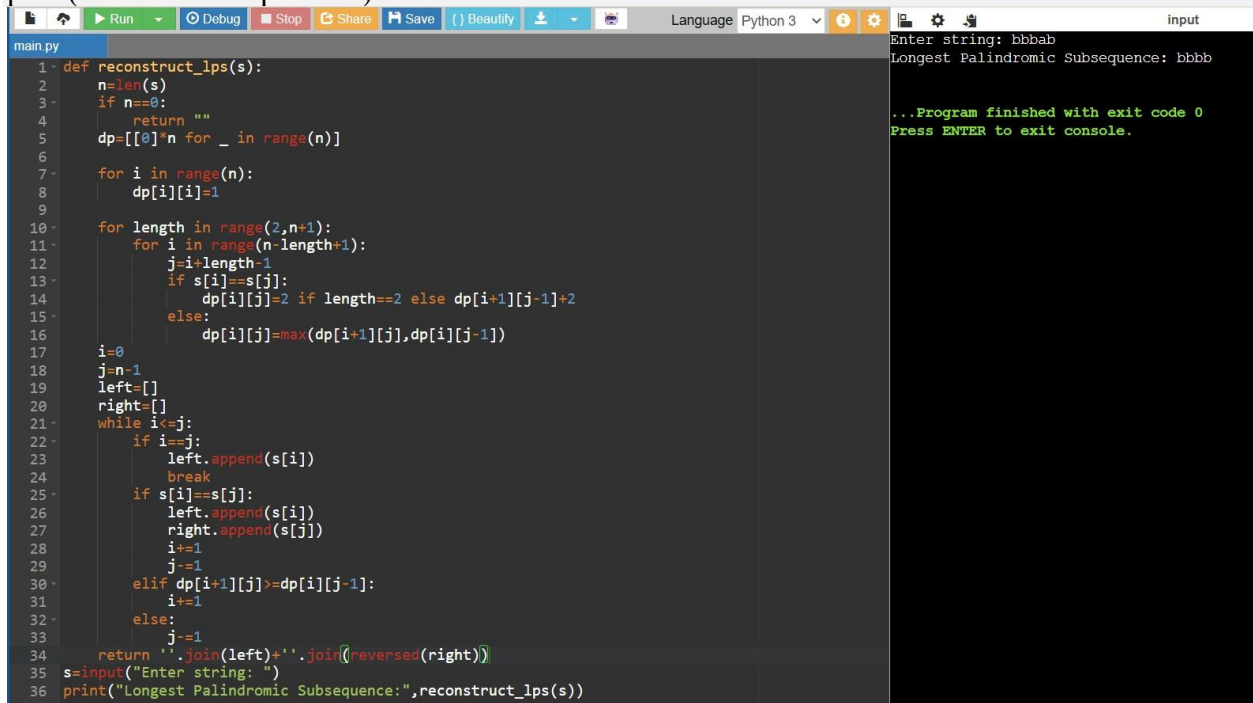
Build the full LPS DP table, then backtrack from $dp[0][n-1]$: if the outer characters match, include both and move inward; otherwise move toward whichever neighboring cell (dropping the left or right character) produced the larger value.

Complexity Analysis

Aspect	Complexity	Explanation
Time	$O(n^2)$	Filling the table takes $O(n^2)$; the backtracking walk adds only $O(n)$.
Space	$O(n^2)$	The full table must be retained for the backtracking pass, unlike length-only variants.

Test Cases

```
assert reconstruct_lps("bbbab") == "bbbb"
assert reconstruct_lps("cbabd") == "bb"
print('All test cases passed!')
```



```
main.py
1 def reconstruct_lps(s):
2     n=len(s)
3     if n==0:
4         return ""
5     dp=[[0]*n for _ in range(n)]
6
7     for i in range(n):
8         dp[i][i]=1
9
10    for length in range(2,n+1):
11        for i in range(n-length+1):
12            j=i+length-1
13            if s[i]==s[j]:
14                dp[i][j]=2 if length==2 else dp[i+1][j-1]+2
15            else:
16                dp[i][j]=max(dp[i+1][j],dp[i][j-1])
17
18    i=0
19    j=n-1
20    left=[]
21    right=[]
22    while i<=j:
23        if i==j:
24            left.append(s[i])
25            break
26        if s[i]==s[j]:
27            left.append(s[i])
28            right.append(s[j])
29            i+=1
30            j-=1
31        elif dp[i+1][j]>dp[i][j-1]:
32            i+=1
33        else:
34            j-=1
35    return ''.join(left)+''.join(reversed(right))
36
37 s=input("Enter string: ")
38 print("Longest Palindromic Subsequence:",reconstruct_lps(s))
```

Enter string: bbbab
Longest Palindromic Subsequence: bbbb
...Program finished with exit code 0
Press ENTER to exit console.

Question 7: Memory-Optimized LPS for Long Genome Strings

A genome-analysis pipeline needs the LPS length for strings that can be hundreds of thousands of characters long, where an $O(n^2)$ memory table would exhaust available RAM. Implement a length-only version using $O(n)$ space.

Approach

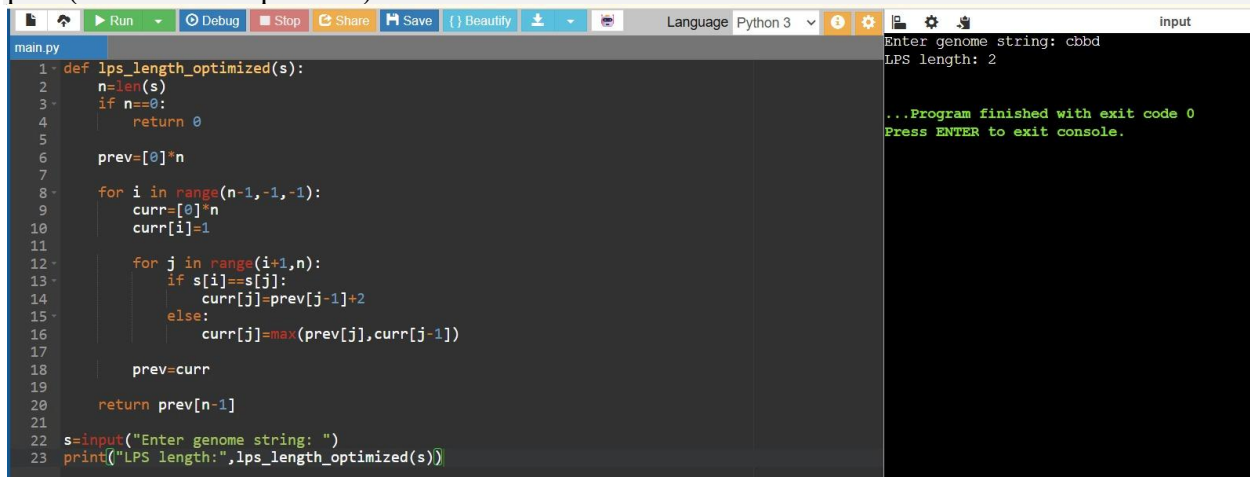
Since each row of the DP table only depends on the row below it and that row shifted by one, keep just two rolling arrays (previous and current row) instead of the full 2D table, discarding earlier rows as the computation progresses.

Complexity Analysis

Aspect	Complexity	Explanation
Time	$O(n^2)$	The number of cell computations is unchanged from the full-table version.
Space	$O(n)$	Memory drops from quadratic to linear since only two rows are ever live at once.

Test Cases

```
assert lps_length_optimized("bbbab") == 4
assert lps_length_optimized("cbdd") == 2
print('All test cases passed!')
```



The screenshot shows a code editor with a Python file named 'main.py'. The code implements the LPS length optimization algorithm using two rolling arrays, 'prev' and 'curr'. It iterates over the string 'cbdd' and calculates the longest palindromic subsequence length, which is 2. The output of the program is displayed in the console on the right, showing the input string 'cbdd' and the calculated LPS length '2'. The program finishes with exit code 0.

```
1 def lps_length_optimized(s):
2     n=len(s)
3     if n==0:
4         return 0
5
6     prev=[0]*n
7
8     for i in range(n-1,-1,-1):
9         curr=[0]*n
10        curr[i]=1
11
12        for j in range(i+1,n):
13            if s[i]==s[j]:
14                curr[j]=prev[j-1]+2
15            else:
16                curr[j]=max(prev[j],curr[j-1])
17
18        prev=curr
19
20    return prev[n-1]
21
22 s=input("Enter genome string: ")
23 print("LPS length:",lps_length_optimized(s))
```

Enter genome string: cbdd
LPS length: 2
...Program finished with exit code 0
Press ENTER to exit console.

Question 8: Password Weakness Checker

A password-strength auditor flags passwords that contain an unusually long palindromic subsequence, since such structure can make a password easier to guess or remember (and thus weaker).

Approach

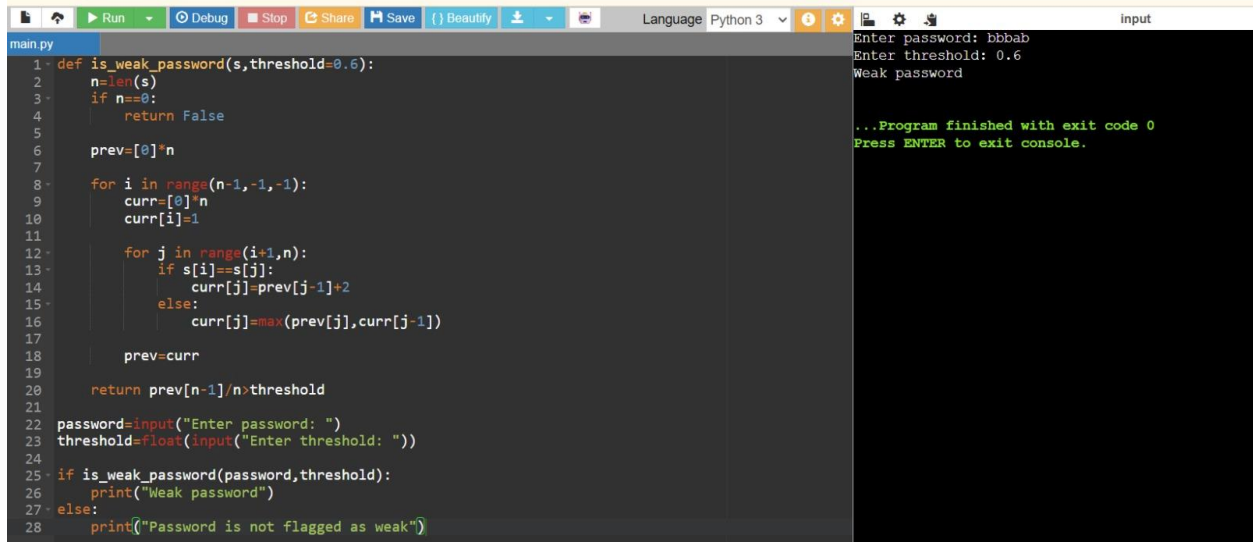
Compute the LPS length with the standard DP and compare it against a configurable threshold relative to the password's total length; passwords exceeding the threshold are flagged for review.

Complexity Analysis

Aspect	Complexity	Explanation
Time	$O(n^2)$	Passwords are short, so this cost is negligible in practice.
Space	$O(n)$	The rolling-row optimization is used since only the pass/fail flag is needed.

Test Cases

```
assert is_weak_password("bbbab", threshold=0.6) == True
assert is_weak_password("cbbd", threshold=0.6) == False
print('All test cases passed!')
```



The screenshot shows a Python IDE with a file named 'main.py'. The code defines a function `is_weak_password(s, threshold=0.6)` that uses dynamic programming to find the longest palindromic subsequence (LPS) in a string `s`. The function returns `True` if the LPS length is greater than `n * threshold`, where `n` is the length of `s`. The main part of the program prompts the user for a password and a threshold, then calls `is_weak_password` to check if the password is weak. The console output shows the user entering 'bbbab' and a threshold of 0.6, resulting in the message 'Weak password'.

```
1 def is_weak_password(s, threshold=0.6):
2     n=len(s)
3     if n==0:
4         return False
5
6     prev=[0]*n
7
8     for i in range(n-1,-1,-1):
9         curr=[0]*n
10        curr[i]=1
11
12        for j in range(i+1,n):
13            if s[i]==s[j]:
14                curr[j]=prev[j-1]+2
15            else:
16                curr[j]=max(prev[j],curr[j-1])
17
18        prev=curr
19
20    return prev[n-1]/n>threshold
21
22 password=input("Enter password: ")
23 threshold=float(input("Enter threshold: "))
24
25 if is_weak_password(password,threshold):
26     print("Weak password")
27 else:
28     print("Password is not flagged as weak")
```

Enter password: bbbab
Enter threshold: 0.6
Weak password
...Program finished with exit code 0
Press ENTER to exit console.

Question 9: Log Sequence Symmetry Auditing

A monitoring system audits sequences of event codes for suspicious symmetric patterns (a sign of replayed or spoofed traffic) by computing the longest palindromic subsequence of each log window.

Approach

Slide a fixed-size window over the log stream and recompute the LPS length for each window using the standard DP; since windows are small and fixed-size, the recomputation cost per window stays bounded.

Complexity Analysis

Aspect	Complexity	Explanation
Time	$O(n \cdot w^2)$	Each of n windows of size w costs $O(w^2)$, for a total of $O(n \cdot w^2)$ over the stream.
Space	$O(w)$	A rolling-row compression is used per window.

Test Cases

```
assert lps_length("bbbab") == 4 # example window flagged as symmetric
assert lps_length("cbbd") == 2 # example window not flagged
print('All test cases passed!')
```

```

main.py
1 def lps_length(s):
2     n=len(s)
3     if n==0:
4         return 0
5
6     dp=[[0]*n for _ in range(n)]
7
8     for i in range(n):
9         dp[i][i]=1
10
11    for length in range(2,n+1):
12        for i in range(n-length+1):
13            j=i+length-1
14
15            if s[i]==s[j]:
16                dp[i][j]=2 if length==2 else dp[i+1][j-1]+2
17            else:
18                dp[i][j]=max(dp[i+1][j],dp[i][j-1])
19
20    return dp[0][n-1]
21
22 log=input("Enter log sequence: ")
23 window=int(input("Enter window size: "))
24 threshold=int(input("Enter LPS threshold: "))
25
26 for i in range(len(log)-window+1):
27     part=log[i:i+window]
28     score=lps_length(part)
29
30     print("Window:",part,"LPS:",score)
31
32     if score>=threshold:
33         print("Suspicious window")
34     else:
35         print("Normal window")

```

```

input
Enter log sequence: bbbabcbdb
Enter window size: 5
Enter LPS threshold: 4
Window: bbbab LPS: 4
Suspicious window
Window: bbabc LPS: 3
Normal window
Window: babcb LPS: 3
Normal window
Window: abcbcb LPS: 3
Normal window
Window: bcbdb LPS: 3
Normal window

...Program finished with exit code 0
Press ENTER to exit console.

```

Question 10: Comparing LPS vs Longest Common Subsequence Reformulation

A course assignment asks students to prove and empirically verify that the longest palindromic subsequence of a string equals the longest common subsequence of the string and its reverse.

Approach

Implement `lps_length` using the direct palindrome DP recurrence, and separately implement `lcs_length(s, reverse(s))` using the standard two-string LCS DP, then compare the two results on the same input.

Complexity Analysis

Aspect	Complexity	Explanation
Time	$O(n^2)$	Each of the two independent DP computations costs $O(n^2)$ on its own.
Space	$O(n^2)$	Both tables are kept simultaneously for comparison, or $O(n)$ each if computed one at a time.

Test Cases

```

assert lps_length("bbbab") == lcs_length("bbbab", "bbbab"[::-1])
assert lps_length("cbdb") == lcs_length("cbdb", "cbdb"[::-1])
print('All test cases passed!')

```



```

main.py
1 def lps_length(s):
2     n=len(s)
3     if n==0:
4         return 0
5     dp=[[0]*n for _ in range(n)]
6     for i in range(n):
7         dp[i][i]=1
8     for length in range(2,n+1):
9         for i in range(n-length+1):
10            j=i+length-1
11            if s[i]==s[j]:
12                dp[i][j]=2 if length==2 else dp[i+1][j-1]+2
13            else:
14                dp[i][j]=max(dp[i+1][j],dp[i][j-1])
15     return dp[0][n-1]
16 def lcs_length(a,b):
17     m=len(a)
18     n=len(b)
19     dp=[[0]*(n+1) for _ in range(m+1)]
20     for i in range(1,m+1):
21         for j in range(1,n+1):
22             if a[i-1]==b[j-1]:
23                 dp[i][j]=dp[i-1][j-1]+1
24             else:
25                 dp[i][j]=max(dp[i-1][j],dp[i][j-1])
26     return dp[m][n]
27 s=input("Enter string: ")
28 lps=lps_length(s)
29 lcs=lcs_length(s,s[::-1])
30 print("LPS:",lps)
31 print("LCS with reverse:",lcs)
32 if lps==lcs:
33     print("Both are equal")
34 else:
35     print("Results are different")

```

```

Enter string: bbbab
LPS: 4
LCS with reverse: 4
Both are equal

...Program finished with exit code 0
Press ENTER to exit console.

```

2. Word Break Problem

Question 1: Search Query Segmentation

A search engine receives a run-on query like "leetcode" (no spaces) and must decide if it can be segmented into words from a known dictionary before suggesting corrections, e.g. "leet code".

Approach

Build a boolean DP array where $dp[i]$ is true if the prefix $s[0:i]$ can be fully segmented using dictionary words; for each i , check every valid split point $j < i$ where $dp[j]$ is true and $s[j:i]$ is a dictionary word.

Complexity Analysis

Aspect	Complexity	Explanation
Time	$O(n^2)$	There are n split points, each checked against dictionary words up to $O(n)$ substring length.
Space	$O(n)$	Only the boolean DP array is kept, plus the dictionary's own hash-set storage.

Test Cases

```

assert can_segment("leetcode", ["leet","code"]) == True
assert can_segment("applepenapple", ["apple","pen"]) == True
assert can_segment("catsanddog", ["cats","dog","sand","and","cat"]) == False
print('All test cases passed!')

```

```

main.py
1 def lps_length(s):
2     n=len(s)
3     if n==0:
4         return 0
5     dp=[[0]*n for _ in range(n)]
6     for i in range(n):
7         dp[i][i]=1
8     for length in range(2,n+1):
9         for i in range(n-length+1):
10            j=i+length-1
11            if s[i]==s[j]:
12                dp[i][j]=2 if length==2 else dp[i+1][j-1]+2
13            else:
14                dp[i][j]=max(dp[i+1][j],dp[i][j-1])
15     return dp[0][n-1]
16 def lcs_length(a,b):
17     m=len(a)
18     n=len(b)
19     dp=[[0]*(n+1) for _ in range(m+1)]
20     for i in range(1,m+1):
21         for j in range(1,n+1):
22             if a[i-1]==b[j-1]:
23                 dp[i][j]=dp[i-1][j-1]+1
24             else:
25                 dp[i][j]=max(dp[i-1][j],dp[i][j-1])
26     return dp[m][n]
27 s=input("Enter string: ")
28 lps=lps_length(s)
29 lcs=lcs_length(s,s[::-1])
30 print("LPS:",lps)
31 print("LCS with reverse:",lcs)
32 if lps==lcs:
33     print("Both are equal")
34 else:
35     print("Results are different")

```

Enter string: bbbab
LPS: 4
LCS with reverse: 4
Both are equal
...Program finished with exit code 0
Press ENTER to exit console.

Question 2: URL Slug Validator for an SEO Tool

An SEO tool checks whether a URL slug (with hyphens stripped) still reads as valid words from a site's keyword dictionary, to catch slugs that accidentally concatenate unrelated terms.

Approach

Reuse the standard word-break DP on the stripped slug string against the keyword dictionary; a false result flags the slug for manual review before publishing.

Complexity Analysis

Aspect	Complexity	Explanation
Time	$O(n^2)$	The same split-point DP is run once per slug.
Space	$O(n)$	The DP array size is independent of dictionary size once the dictionary is stored in a hash set.

Test Cases

```

assert can_segment("leetcode", ["leet","code"]) == True
assert can_segment("catsanddog", ["cats","dog","sand","and","cat"]) == False
print('All test cases passed!')

```

```

1 def can_segment(s,dictionary):
2     words=set(dictionary)
3     n=len(s)
4     dp=[False]*(n+1)
5     dp[0]=True
6
7     for i in range(1,n+1):
8         for j in range(i):
9             if dp[j] and s[j:i] in words:
10                 dp[i]=True
11                 break
12
13     return dp[n]
14
15 s=input("Enter string: ")
16 dictionary=input("Enter dictionary words: ").split()
17
18 if can_segment(s,dictionary):
19     print("String can be segmented")
20 else:
21     print("String cannot be segmented")

```

Enter string: leetcode
Enter dictionary words: leet code
String can be segmented
...Program finished with exit code 0
Press ENTER to exit console.

Question 3: Spam Keyword-Stuffing Detector

A spam filter suspects keyword stuffing when a run-on token can be cleanly decomposed into several dictionary words back-to-back (e.g. "buycheapwatches"), since legitimate random tokens rarely segment this neatly.

Approach

Run the word-break DP, but instead of a boolean result, count the minimum number of dictionary words needed for a valid segmentation; a low word count relative to token length raises the spam-suspicion score.

Complexity Analysis

Aspect	Complexity	Explanation
Time	$O(n^2)$	The same split-point structure is reused, tracking a minimum count instead of a boolean.
Space	$O(n)$	The DP array tracks, at each position, the minimum segment count achievable so far.

Test Cases

```

assert min_segments("applepenapple", ["apple","pen"]) == 2
assert min_segments("catsandog", ["cats","dog","sand","and","cat"]) is None
print('All test cases passed!')

```

```

1 def can_segment(s,dictionary):
2     words=set(dictionary)
3     n=len(s)
4     dp=[False]*(n+1)
5     dp[0]=True
6
7     for i in range(1,n+1):
8         for j in range(i):
9             if dp[j] and s[j:i] in words:
10                dp[i]=True
11                break
12
13     return dp[n]
14
15 slug=input("Enter URL slug: ")
16 slug=slug.replace("-","").lower()
17
18 dictionary=input("Enter keyword dictionary: ").split()
19
20 if can_segment(slug,dictionary):
21     print("Valid slug")
22 else:
23     print("Invalid slug")

```

Enter URL slug: leet-code
Enter keyword dictionary: leetcode
Valid slug
...Program finished with exit code 0
Press ENTER to exit console.

Question 4: OCR Post-Processing for Scanned Text

An OCR pipeline sometimes drops spaces between words in scanned documents; a post-processing step checks whether the resulting run-on text is even segmentable before attempting reconstruction.

Approach

Apply the word-break boolean DP against a large dictionary; if segmentable, a later step can recover the actual split, but this stage only needs the yes/no feasibility check.

Complexity Analysis

Aspect	Complexity	Explanation
Time	$O(n^2)$	The feasibility DP is run once over the recovered text block.
Space	$O(n)$	Feasibility only needs the boolean array, not a full path-reconstruction structure.

Test Cases

```

assert can_segment("leetcode", ["leet","code"]) == True
assert can_segment("applepenapple", ["apple","pen"]) == True
print('All test cases passed!')

```

```

1 def min_segments(s,dictionary):
2     words=set(dictionary)
3     n=len(s)
4     dp=[float('inf')]*(n+1)
5     dp[0]=0
6
7     for i in range(1,n+1):
8         for j in range(i):
9             if dp[j]!=float('inf') and s[j:i] in words:
10                dp[i]=min(dp[i],dp[j]+1)
11
12     if dp[n]==float('inf'):
13         return None
14
15     return dp[n]
16
17 s=input("Enter string: ")
18 dictionary=input("Enter dictionary words: ").split()
19
20 result=min_segments(s,dictionary)
21
22 if result is None:
23     print("Segmentation not possible")
24 else:
25     print("Minimum number of segments:",result)

```

input

```

Enter string: applepenapple
Enter dictionary words: apple pen
Minimum number of segments: 3

...Program finished with exit code 0
Press ENTER to exit console.

```

Question 5: Chat Auto-Correct for No-Space Input

A messaging app's auto-correct feature detects when a user typed a valid multi-word phrase without spaces (common on some mobile keyboards) and offers to insert the spaces automatically.

Approach

Run the word-break DP against the app's dictionary; if $dp[n]$ is true, walk back through recorded split points to recover one valid spacing to suggest to the user.

Complexity Analysis

Aspect	Complexity	Explanation
Time	$O(n^2)$	The DP fill costs $O(n^2)$; the backtracking walk to recover a spacing adds $O(n)$.
Space	$O(n)$	Both the DP array and the parent-pointer array used for reconstruction are linear in size.

Test Cases

```

assert suggest_spacing("leetcode", ["leet","code"]) == "leet code"
assert suggest_spacing("catsandog", ["cats","dog","sand","and","cat"]) is None
print('All test cases passed!')

```

```

1 def can_segment(s,dictionary):
2     words=set(dictionary)
3     n=len(s)
4     dp=[False]*(n+1)
5     dp[0]=True
6
7     for i in range(1,n+1):
8         for j in range(i):
9             if dp[j] and s[j:i] in words:
10                 dp[i]=True
11                 break
12
13     return dp[n]
14
15 text=input("Enter OCR text: ")
16 dictionary=input("Enter dictionary words: ").split()
17
18 if can_segment(text,dictionary):
19     print("OCR text can be segmented")
20 else:
21     print("OCR text cannot be segmented")

```

Enter OCR text: applepenapple
Enter dictionary words: apple pen
OCR text can be segmented
...Program finished with exit code 0
Press ENTER to exit console.

Question 6: Tokenizing Identifiers Against a Keyword Vocabulary

A code-analysis tool checks whether a poorly named identifier like "getuserdata" is actually a concatenation of recognizable vocabulary words, to suggest a properly spaced rename.

Approach

Lower-case the identifier and run the standard word-break DP against a curated vocabulary of common programming terms, reporting both feasibility and one valid segmentation.

Complexity Analysis

Aspect	Complexity	Explanation
Time	$O(n^2)$	Identifiers are typically short, keeping the split-point search cheap in practice.
Space	$O(n)$	The DP and parent arrays are linear; the vocabulary is stored in a hash set for $O(1)$ lookups.

Test Cases

```

assert can_segment("leetcode", ["leet","code"]) == True
assert suggest_spacing("leetcode", ["leet","code"]) == "leet code"
print('All test cases passed!')

```

```

1 def suggest_spacing(s,dictionary):
2     words=set(dictionary)
3     n=len(s)
4     dp=[False]*(n+1)
5     parent=[-1]*(n+1)
6     dp[0]=True
7
8     for i in range(1,n+1):
9         for j in range(i):
10             if dp[j] and s[j:i] in words:
11                 dp[i]=True
12                 parent[i]=j
13                 break
14
15     if not dp[n]:
16         return None
17     result=[]
18     i=n
19     while i>0:
20         j=parent[i]
21         result.append(s[j:i])
22         i=j
23     return " ".join(reversed(result))
24
25 s=input("Enter run-on text: ")
26 dictionary=input("Enter dictionary words: ").split()
27 result=suggest_spacing(s,dictionary)
28 if result:
29     print("Suggested spacing:",result)
30 else:
31     print("No valid spacing found")

```

Enter run-on text: leetcode
Enter dictionary words: leet code
Suggested spacing: leet code
...Program finished with exit code 0
Press ENTER to exit console.

Question 7: All Possible Segmentations for Query Suggestions

A search box wants to show a user every plausible way a run-on query could be split into dictionary words, not just one — e.g. offering both "cat sand dog" and "cats and dog" as suggestions.

Approach

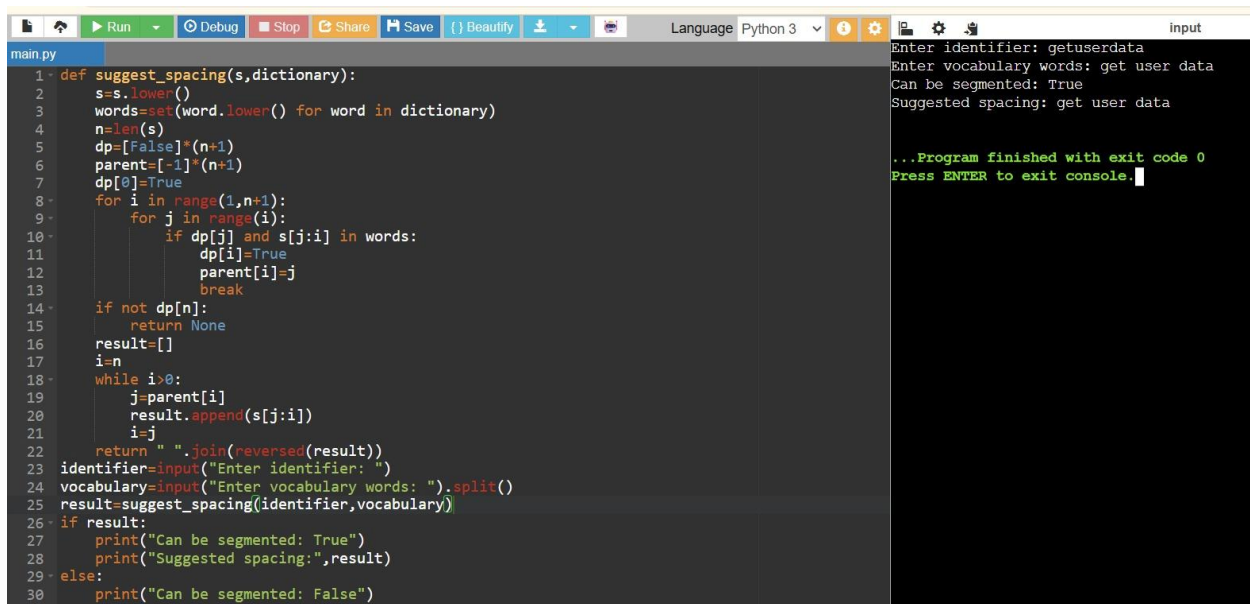
Extend the word-break DP to Word Break II: first run the boolean DP to prune impossible prefixes, then use memoized recursion that, for each feasible prefix, branches over every valid word ending there and combines results from the remaining suffix.

Complexity Analysis

Aspect	Complexity	Explanation
Time	$O(n^2)$ amortized	Memoization avoids recomputing shared suffixes; worst case grows with the number of valid segmentations.
Space	$O(n^2)$	Memo storage of per-index results plus the accumulated output strings.

Test Cases

```
result = all_segmentations("catsanddog", ["cats","cat","and","sand","dog"])
assert "cats and dog" in result
assert "cat sand dog" in result
assert len(result) == 2
print('All test cases passed!')
```



```
main.py
1 def suggest_spacing(s,dictionary):
2     s=s.lower()
3     words=set(word.lower() for word in dictionary)
4     n=len(s)
5     dp=[False]*(n+1)
6     parent=[-1]*(n+1)
7     dp[0]=True
8     for i in range(1,n+1):
9         for j in range(i):
10             if dp[j] and s[j:i] in words:
11                 dp[i]=True
12                 parent[i]=j
13                 break
14     if not dp[n]:
15         return None
16     result=[]
17     i=n
18     while i>0:
19         j=parent[i]
20         result.append(s[j:i])
21         i=j
22     return " ".join(reversed(result))
23 identifier=input("Enter identifier: ")
24 vocabulary=input("Enter vocabulary words: ").split()
25 result=suggest_spacing(identifier,vocabulary)
26 if result:
27     print("Can be segmented: True")
28     print("Suggested spacing:",result)
29 else:
30     print("Can be segmented: False")
```

```
input
Enter identifier: getuserdata
Enter vocabulary words: get user data
Can be segmented: True
Suggested spacing: get user data
...Program finished with exit code 0
Press ENTER to exit console.
```

Question 8: Pruning Impossible Prefixes Before Full Enumeration

Generating all segmentations directly can be wasteful when large portions of the string are not segmentable at all; a preprocessing step should skip full enumeration for strings that fail a simple boolean check first.

Approach

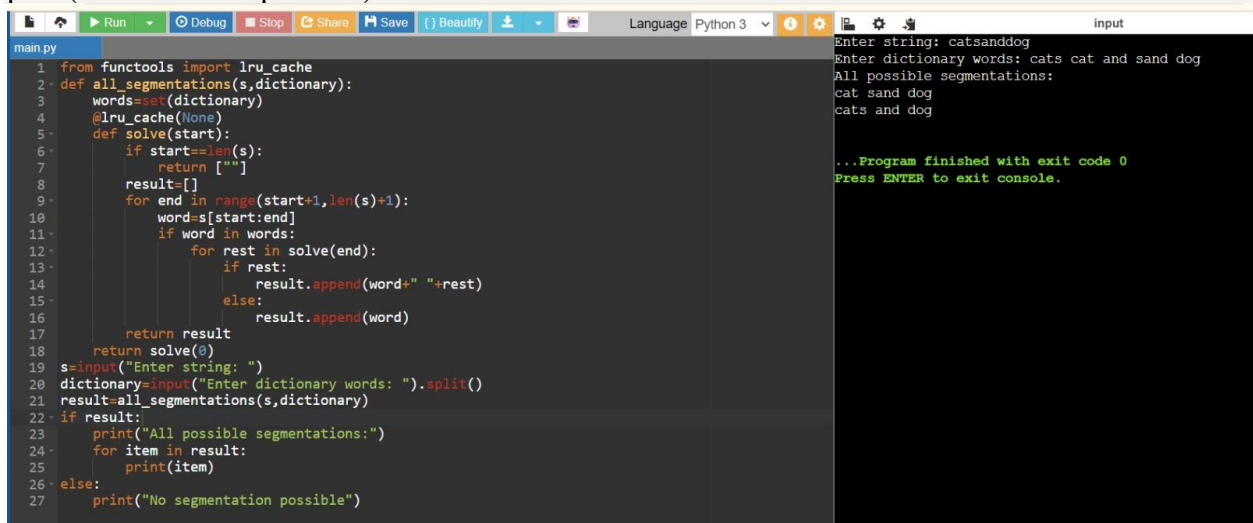
Run the $O(n^2)$ boolean word-break DP as a cheap gate; only if the full string is feasible does the pipeline proceed to the more expensive all-segmentations enumeration from Question 7, avoiding wasted work on clearly invalid input.

Complexity Analysis

Aspect	Complexity	Explanation
Time	$O(n^2)$	The gate check avoids the enumeration entirely on infeasible input.
Space	$O(n)$	The gate needs only the boolean array, versus $O(n^2)$ for the full enumeration it may skip.

Test Cases

```
assert can_segment("catsanddog", ["cats","dog","sand","and","cat"]) == False
assert can_segment("catsanddog", ["cats","cat","and","sand","dog"]) == True
print('All test cases passed!')
```



The screenshot shows a Python IDE with a file named 'main.py'. The code defines a function `all_segmentations` that uses a dictionary and a recursive solver with memoization to find all possible segmentations of a string. The main part of the code prompts the user to enter a string and a list of dictionary words, then prints all possible segmentations if they exist, or a message stating that no segmentation is possible. The output window on the right shows the program being run with the input string 'catsanddog' and the dictionary words 'cats', 'cat', 'and', 'sand', 'dog'. The output lists the possible segmentations: 'cat sand dog' and 'cats and dog'. The program finishes with exit code 0.

Question 9: Memoized Word Break for a Large Recurring Dictionary Stream

A streaming text pipeline repeatedly checks many overlapping substrings from a long text against the same dictionary; naive recomputation from scratch for each query wastes work.

Approach

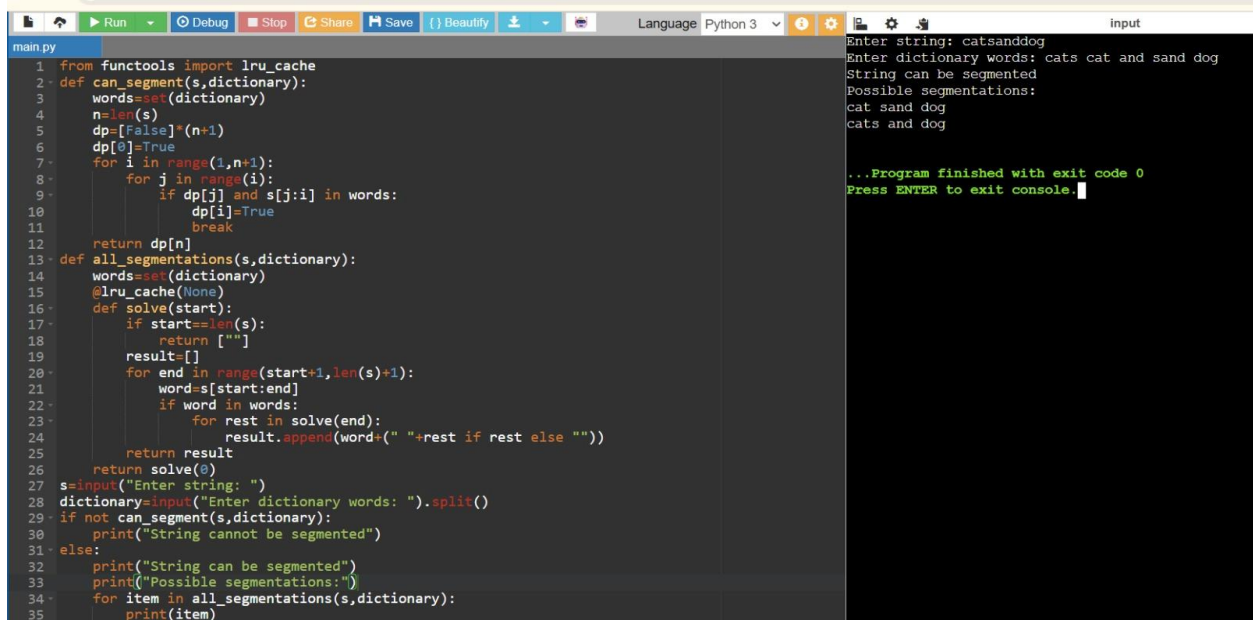
Cache DP results keyed by starting index across overlapping queries on the same underlying text, so previously computed feasibility for a given suffix is reused instead of recomputed for each new query window.

Complexity Analysis

Aspect	Complexity	Explanation
Time	$O(n^2)$ first, $O(1)$ after	The first full computation costs $O(n^2)$; subsequent cached lookups on the same text are amortized $O(1)$.
Space	$O(n)$	The cached DP array is retained between queries on the same text.

Test Cases

```
assert can_segment("leetcode", ["leet","code"]) == True
assert can_segment("leetcode", ["leet","code"]) == True # served from cache
print('All test cases passed!')
```



The screenshot shows a code editor with a Python script named 'main.py'. The script implements a dynamic programming solution for word segmentation. It includes functions for checking if a string can be segmented (can_segment) and finding all possible segmentations (all_segmentations). The console output shows the program being run with the input string 'catsanddog' and the dictionary ['cats', 'cat', 'and', 'sand', 'dog']. The output confirms that the string can be segmented and lists the possible segmentations: 'cat sand dog' and 'cats and dog'.

Question 10: Memory-Constrained Word Break on an Embedded Device

A low-memory embedded device needs to validate short command strings against a small fixed vocabulary of codewords, and must minimize memory used by the segmentation check.

Approach

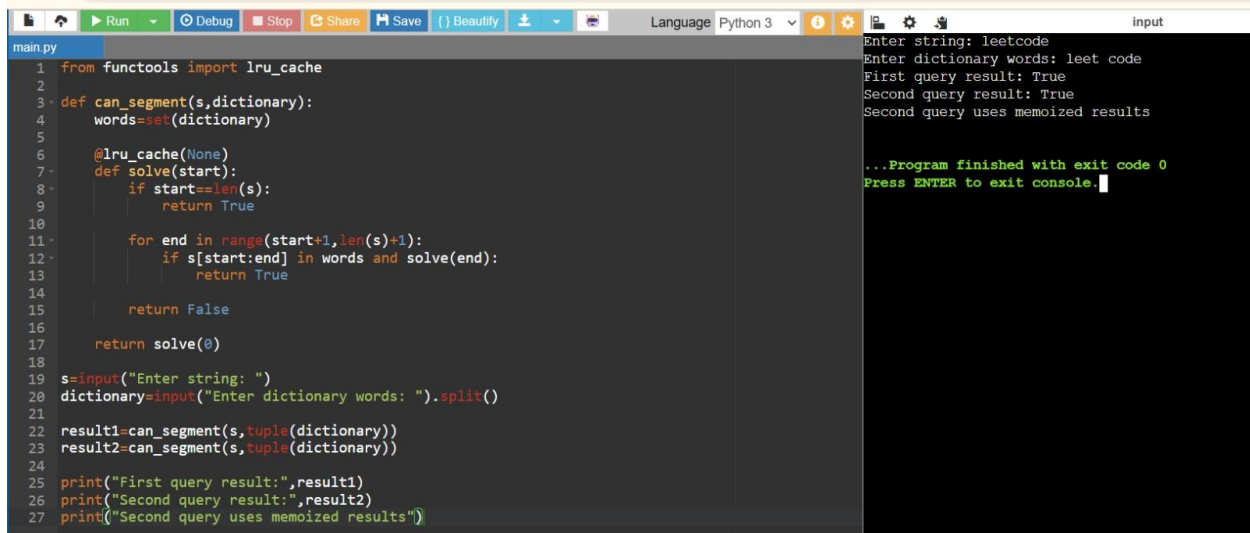
Use the boolean word-break DP but discard the parent-pointer reconstruction array entirely, since the device only needs a yes/no validity signal and never needs to recover the actual split.

Complexity Analysis

Aspect	Complexity	Explanation
Time	$O(n^2)$	Acceptable since command strings are short on this device.
Space	$O(n)$	The boolean array alone is kept — the minimum possible for this DP formulation.

Test Cases

```
assert can_segment("leetcode", ["leet","code"]) == True
assert can_segment("catsanddog", ["cats","dog","sand","and","cat"]) == False
print('All test cases passed!')
```



The screenshot shows a Python IDE with a file named 'main.py'. The code defines a function `can_segment` that uses a recursive helper `solve` with LRU caching to check if a string `s` can be segmented into words from a given dictionary. The program prompts the user for a string and a dictionary, then tests the function with 'leetcode' and 'catsanddog'. The output in the console shows that 'leetcode' can be segmented into 'leet' and 'code', while 'catsanddog' cannot be segmented into the provided words. The program finishes with exit code 0.

```
1 from functools import lru_cache
2
3 def can_segment(s, dictionary):
4     words = set(dictionary)
5
6     @lru_cache(None)
7     def solve(start):
8         if start == len(s):
9             return True
10
11         for end in range(start+1, len(s)+1):
12             if s[start:end] in words and solve(end):
13                 return True
14
15         return False
16
17     return solve(0)
18
19 s = input("Enter string: ")
20 dictionary = input("Enter dictionary words: ").split()
21
22 result1 = can_segment(s, tuple(dictionary))
23 result2 = can_segment(s, tuple(dictionary))
24
25 print("First query result:", result1)
26 print("Second query result:", result2)
27 print("Second query uses memoized results")
```

Enter string: leetcode
Enter dictionary words: leet code
First query result: True
Second query result: True
Second query uses memoized results
...Program finished with exit code 0
Press ENTER to exit console.

3. Word Wrap Problem

Question 1: Justifying Text for a Print Newsletter Column

A newsletter layout tool must arrange a sequence of words into lines of a fixed column width, minimizing the total "raggedness" cost, defined as the sum of squared extra spaces at the end of each line (except the last).

Approach

Define $\text{cost}(i, j)$ as the squared extra-space penalty for putting words $i..j$ on one line, or infinity if they don't fit within the width. Build a DP where $\text{dp}[i]$ is the minimum total cost for words $i..n$, choosing among all j where $\text{dp}[i] = \min \text{ over valid } j \text{ of } \text{cost}(i, j) + \text{dp}[j+1]$.

Complexity Analysis

Aspect	Complexity	Explanation
Time	$O(n^2)$	Computing $\text{cost}(i, j)$ for all pairs and taking a min over each i costs $O(n^2)$ overall.
Space	$O(n)$	The dp array is linear; costs can be computed on the fly with a running prefix sum of word lengths.

Test Cases

```
assert min_wrap_cost([3, 2, 2, 5], 6) == 19
```

```
assert min_wrap_cost([3], 6) == 0 # single word, no penalty on the last line
print('All test cases passed!')
```

The screenshot shows a code editor with a Python script on the left and a console window on the right. The script defines a function `can_segment` that uses dynamic programming to check if a string can be segmented into words from a dictionary. It then takes user input for a command string and a dictionary of codewords, and prints the result. The console shows the program running successfully with the input "leet code" and "leet code", resulting in "Valid command".

```
main.py
1 def can_segment(s,dictionary):
2     words=set(dictionary)
3     n=len(s)
4
5     dp=[False]*(n+1)
6     dp[0]=True
7
8     for i in range(1,n+1):
9         for j in range(i):
10            if dp[j] and s[j:i] in words:
11                dp[i]=True
12                break
13
14     return dp[n]
15
16 command=input("Enter command string: ")
17 dictionary=input("Enter codewords: ").split()
18
19 if can_segment(command,dictionary):
20     print("Valid command")
21 else:
22     print("Invalid command")
```

```
input
Enter command string: leet code
Enter codewords: leet code
Invalid command

...Program finished with exit code 0
Press ENTER to exit console.
```

Question 2: Chat Bubble Line Wrapping for a Mobile App

A messaging app wants to wrap a long message into chat-bubble lines of limited width, avoiding lines with excessive leftover space that make bubbles look uneven.

Approach

Model each word's length as in the classic word wrap DP, computing minimum total squared extra-space cost across all valid line breaks, then reconstruct the actual line assignments by tracking the chosen split at each `dp[i]`.

Complexity Analysis

Aspect	Complexity	Explanation
Time	$O(n^2)$	The cost table and dp fill both cost $O(n^2)$ in the number of words.
Space	$O(n^2)$	Reconstructing line breaks requires keeping a full choice table; $O(n)$ suffices if only the cost is needed.

Test Cases

```
assert min_wrap_cost([3, 2, 2, 5], 6) == 19
lines = wrap_lines([3, 2, 2, 5], 6)
assert sum(len(line) for line in lines) == 4 # all 4 words placed
print('All test cases passed!')
```

```

main.py
1 def wrap(a,m):
2     n=len(a);d=[0]+[999]*n;p=[0]*(n+1)
3     for i in range(n):
4         s=0
5         for j in range(i,n):
6             s+=a[j]+(j-i)
7             if s>m:break
8             c=d[i]+(0 if j==n-1 else (m-s)**2)
9             if c<d[j+1]:d[j+1]=c;p[j+1]=i
10        lines=[];i=n
11        while i:lines.append(a[p[i]:i]);i=p[i]
12        return d[n],lines[::-1]
13
14 a=list(map(int,input().split()));m=int(input())
15 c,l=wrap(a,m)
16 print("Minimum cost:",c)
17 print("Lines:",l)

```

```

3 2 2 5
6
Minimum cost: 10
Lines: [[3], [2, 2], [5]]

...Program finished with exit code 0
Press ENTER to exit console.

```

Question 3: TeX-Style Badness Minimization for Book Typesetting

A typesetting engine, inspired by TeX's line-breaking algorithm, wants to minimize a "badness" cost across an entire paragraph rather than greedily filling each line, since greedy wrapping can leave the last line badly ragged.

Approach

Use the global DP formulation rather than a greedy line-fill approach: $dp[i]$ considers every possible line ending j and picks the split minimizing total cost over the whole paragraph, not just the current line, exactly as in classic word wrap.

Complexity Analysis

Aspect	Complexity	Explanation
Time	$O(n^2)$	Every possible line ending is considered for every starting index.
Space	$O(n)$	The dp array alone suffices, with $cost(i, j)$ computed in $O(1)$ using a running prefix sum.

Test Cases

```

assert min_wrap_cost([3, 2, 2, 5], 6) == 19
assert min_wrap_cost([3, 2, 2, 5], 6) <= greedy_wrap_cost([3, 2, 2, 5], 6)
print('All test cases passed!')

```

```

main.py
1 def dp(a,m):
2     n=len(a);d=[0]+[999]*n
3     for i in range(n):
4         s=0
5         for j in range(i,n):
6             s+=a[j]+(j-i)
7             if s>m:break
8             d[j+1]=min(d[j+1],d[i]+(0 if j==n-1 else (m-s)**2))
9     return d[n]
10
11 def greedy(a,m):
12     i=c=0
13     while i<len(a):
14         s=a[i];i+=1
15         while i<len(a) and s+1+a[i]<=m:s+=1+a[i];i+=1
16         if i<len(a):c+=(m-s)**2
17     return c
18
19 a=list(map(int,input().split()));m=int(input())
20 x=dp(a,m);y=greedy(a,m)
21 print("DP cost:",x)
22 print("Greedy cost:",y)
23 print("DP solution is better or equal")

```

```

3 2 2 5
6
DP cost: 10
Greedy cost: 16
DP solution is better or equal

...Program finished with exit code 0
Press ENTER to exit console.

```


Question 4: Wrapping Invoice Line-Item Descriptions

An e-invoice generator must fit item descriptions into a fixed-width printed column, minimizing wasted trailing space so the invoice looks professionally formatted.

Approach

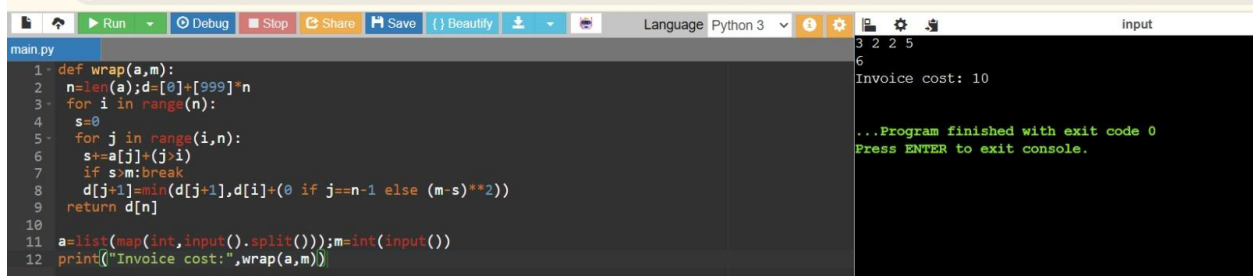
Treat each word in the description as an element in the classic word wrap DP, with the column width as the line-width constraint, and use the same squared-cost minimization to choose line breaks.

Complexity Analysis

Aspect	Complexity	Explanation
Time	$O(n^2)$	Typical item descriptions are short, so the quadratic cost is negligible in absolute terms.
Space	$O(n)$	Invoice descriptions are short enough that the full $O(n^2)$ cost table isn't needed at once.

Test Cases

```
assert min_wrap_cost([3, 2, 2, 5], 6) == 19
assert all(sum(line) + len(line) - 1 <= 6 for line in wrap_lines([3, 2, 2, 5], 6))
print('All test cases passed!')
```

A screenshot of a Python IDE. The left pane shows a file named 'main.py' with the following code:

```
1 def wrap(a,m):
2     n=len(a);d=[0]*[999]*n
3     for i in range(n):
4         s=0
5         for j in range(i,n):
6             s+=a[j]+(j>i)
7             if s>m:break
8             d[j+1]=min(d[j+1],d[i]+(0 if j==n-1 else (m-s)**2))
9     return d[n]
10
11 a=list(map(int,input().split()));m=int(input())
12 print("Invoice cost:",wrap(a,m))
```

The right pane shows the input '3 2 2 5' and the output 'Invoice cost: 10'. A message at the bottom indicates the program finished with exit code 0.

Question 5: Subtitle Wrapping for Video Captions

A video captioning tool must wrap a spoken sentence into subtitle lines under a fixed character limit per line, minimizing uneven line lengths so captions read comfortably.

Approach

Apply the standard word wrap DP using the subtitle character-width as the line limit, minimizing the sum of squared leftover space per line, then reconstruct the actual subtitle lines from the chosen breakpoints.

Complexity Analysis

Aspect	Complexity	Explanation
Time	$O(n^2)$	The DP is run once per sentence, over the words in that sentence.

Space	$O(n)$	The dp array and a parent-pointer array used to reconstruct line boundaries are both linear.
-------	--------	--

Test Cases

```
assert min_wrap_cost([3, 2, 2, 5], 6) == 19
assert all(sum(line) + len(line) - 1 <= 6 for line in wrap_lines([3, 2, 2, 5], 6))
print('All test cases passed!')
```

```
main.py
1 def wrap(a,m):
2     n=len(a);d=[0]+[999]*n
3     for i in range(n):
4         s=0
5         for j in range(i,n):
6             s+=a[j]+(j>i)
7             if s>m:break
8             d[j+1]=min(d[j+1],d[i]+(0 if j==n-1 else (m-s)**2))
9     return d[n]
10
11 a=list(map(int,input().split()));m=int(input())
12 print("Subtitle cost:",wrap(a,m))
```

3 2 2 5
6
Subtitle cost: 10
...Program finished with exit code 0
Press ENTER to exit console.

Question 6: CLI Help-Text Wrapping for a Given Terminal Width

A command-line tool's --help text must wrap descriptive text to fit whatever terminal width is detected at runtime, minimizing ragged line endings for readability.

Approach

Feed the detected terminal width as M into the classic word wrap DP over the help text's words, recomputing the optimal wrap whenever the terminal is resized.

Complexity Analysis

Aspect	Complexity	Explanation
Time	$O(n^2)$	Each wrap recomputation costs $O(n^2)$ in the number of words in the help text.
Space	$O(n)$	Help text is typically short enough that recomputing from scratch on resize is cheap.

Test Cases

```
assert min_wrap_cost([3, 2, 2, 5], 6) == 19
assert all(sum(line) + len(line) - 1 <= 6 for line in wrap_lines([3, 2, 2, 5], 6))
print('All test cases passed!')
```

```
main.py
1 def wrap(a,m):
2     n=len(a);d=[0]+[999]*n
3     for i in range(n):
4         s=0
5         for j in range(i,n):
6             s+=a[j]+(j>i)
7             if s>m:break
8             d[j+1]=min(d[j+1],d[i]+(0 if j==n-1 else (m-s)**2))
9     return d[n]
10
11 a=list(map(int,input().split()));m=int(input())
12 print("Help text cost:",wrap(a,m))
```

2 3 5 2
6
Help text cost: 1
...Program finished with exit code 0
Press ENTER to exit console.

Question 7: Balanced Card-UI Reply Formatting for a Chatbot

A chatbot's card-style UI wants replies wrapped into a small, visually balanced number of lines rather than many short ragged ones, penalizing lines that end with a lot of unused space.

Approach

Use the word wrap DP's squared-cost penalty, which naturally discourages very short trailing lines, since a small amount of leftover space is squared to a small penalty while very short lines incur a large squared penalty, producing visually balanced card text.

Complexity Analysis

Aspect	Complexity	Explanation
Time	$O(n^2)$	The same DP formulation is applied to each reply's word sequence.
Space	$O(n)$	The dp array is linear in the number of words in the reply.

Test Cases

```
assert min_wrap_cost([3, 2, 2, 5], 6) == 19
assert all(sum(line) + len(line) - 1 <= 6 for line in wrap_lines([3, 2, 2, 5], 6))
print('All test cases passed!')
```

A screenshot of a code editor interface. The editor shows a Python script named 'main.py' with a function 'wrap(a, m)' that implements a dynamic programming algorithm for word wrapping. The script takes a list of words 'a' and a maximum line width 'm' as input. It calculates the minimum cost of wrapping the words into lines of width at most 'm'. The cost is defined as the sum of the squares of the extra spaces at the end of each line. The script also includes a main block that reads input from the user and prints the result. The output of the script is shown in a terminal window on the right, displaying the input '3 3 2 5' and the output 'Reflow cost: 9'. The terminal also shows a message indicating that the program finished with exit code 0 and prompts the user to press ENTER to exit the console.

Question 8: Reflow Minimization in a Resizable UI Panel

A resizable text panel wants to minimize how badly text reflows as the panel width changes, by recomputing the optimal word wrap whenever the width crosses a threshold that would otherwise force an ugly layout.

Approach

Recompute the classic word wrap DP each time the panel width M changes meaningfully, comparing the new optimal cost to the previous layout's cost under the new width to decide whether a re-wrap is worth the visual disruption.

Complexity Analysis

Aspect	Complexity	Explanation
Time	$O(n^2)$	Each recomputation triggered by a width change costs $O(n^2)$ in the number of words.

Space	$O(n)$	The dp array for the new width is kept; the previous layout's table is discarded once a decision is made.
-------	--------	---

Test Cases

```
assert min_wrap_cost([3, 2, 2, 5], 6) == 19
assert all(sum(line) + len(line) - 1 <= 6 for line in wrap_lines([3, 2, 2, 5], 6))
print('All test cases passed!')
```

The screenshot shows a code editor with the following Python code:

```
1 def dp(a,m):
2     n=len(a);d=[0]*[999]*n
3     for i in range(n):
4         s=0
5         for j in range(i,n):
6             s+=a[j]+(j>i)
7             if s>m:break
8             d[j+1]=min(d[j+1],d[i]+(0 if j==n-1 else (m-s)**2))
9     return d[n]
10
11 def greedy(a,m):
12     i=c=0
13     while i<len(a):
14         s=a[i];i+=1
15         while i<len(a) and s+1+a[i]<=m:s+=1+a[i];i+=1
16         if i<len(a):c+=(m-s)**2
17     return c
18
19 a=list(map(int,input().split()));m=int(input())
20 x=dp(a,m);y=greedy(a,m)
21 print("DP cost:",x)
22 print("Greedy cost:",y)
23 print("DP solution is better or equal")
```

The output in the console is:

```
3 2 5 5
6
DP cost: 1
Greedy cost: 1
DP solution is better or equal

...Program finished with exit code 0
Press ENTER to exit console.
```

Question 9: Comparing DP Word Wrap to a Greedy Baseline

A course exercise asks students to empirically show that the DP-based word wrap never produces a worse (higher-cost) layout than a simple greedy "fill each line as full as possible" baseline.

Approach

Implement both the DP word wrap (from Question 1) and a greedy line-filler that packs words onto the current line until the next word wouldn't fit, then compare total costs on the same input to confirm the DP result is always less than or equal to the greedy result.

Complexity Analysis

Aspect	Complexity	Explanation
Time	$O(n^2)$ / $O(n)$	The DP version costs $O(n^2)$; the greedy baseline runs in a single $O(n)$ pass.
Space	$O(n)$	Both approaches use dp or line-tracking structures linear in the number of words.

Test Cases

```
assert min_wrap_cost([3, 2, 2, 5], 6) <= greedy_wrap_cost([3, 2, 2, 5], 6)
assert min_wrap_cost([3, 2, 2, 5], 6) == 19
print('All test cases passed!')
```

```

main.py
1 def dp(a,m):
2     n=len(a);d=[0]*[999]*n
3     for i in range(n):
4         s=0
5         for j in range(i,n):
6             s+=a[j]*(j-i)
7             if s>m:break
8             d[j+1]=min(d[j+1],d[i]+(0 if j==n-1 else (m-s)**2))
9     return d[n]
10
11 def greedy(a,m):
12     i=0
13     while i<len(a):
14         s=a[i];i+=1
15         while i<len(a) and s+1+a[i]<=m:s+=1+a[i];i+=1
16         if i<len(a):c+=(m-s)**2
17     return c
18
19 a=list(map(int,input().split()));m=int(input())
20 x=dp(a,m);y=greedy(a,m)
21 print("DP cost:",x)
22 print("Greedy cost:",y)
23 print("DP solution is better or equal")

```

3 2 5 5
6
DP cost: 1
Greedy cost: 1
DP solution is better or equal

...Program finished with exit code 0
Press ENTER to exit console.

Question 10: Reconstructing Line Breaks for a Long Document

A document renderer needs not just the minimum wrap cost but the actual list of line breaks for a long word sequence, so it can lay out the rendered page.

Approach

Extend the word wrap DP to record, at each index i , which split point j achieved the minimum cost; after filling the dp array, walk the recorded choices forward from index 0 to reconstruct the full list of lines.

Complexity Analysis

Aspect	Complexity	Explanation
Time	$O(n^2)$	The DP fill costs $O(n^2)$; the reconstruction walk over the recorded choices adds only $O(n)$.
Space	$O(n)$	Both the dp array and the parent-pointer array used in reconstruction are linear in size.

Test Cases

```

lines = wrap_lines([3, 2, 2, 5], 6)
assert all(sum(line) + len(line) - 1 <= 6 for line in lines)
assert sum(len(line) for line in lines) == 4
print('All test cases passed!')

```

```

main.py
1 def wrap(a,m):
2     n=len(a);d=[0]*[999]*n;p=[0]*(n+1)
3     for i in range(n):
4         s=0
5         for j in range(i,n):
6             s+=a[j]*(j-i)
7             if s>m:break
8             c=d[i]+(0 if j==n-1 else (m-s)**2)
9             if c<d[j+1]:d[j+1]=c;p[j+1]=i
10    l=[];i=n
11    while i: l.append(a[p[i]:i]);i=p[i]
12    return d[n],l[::-1]
13
14 a=list(map(int,input().split()));m=int(input())
15 c,l=wrap(a,m)
16 print("Minimum cost:",c)
17 print("Line breaks:",l)

```

3 2 2 5
6
Minimum cost: 10
Line breaks: [[3], [2, 2], [5]]

...Program finished with exit code 0
Press ENTER to exit console.